

Documentation Conception

GL10
DOC

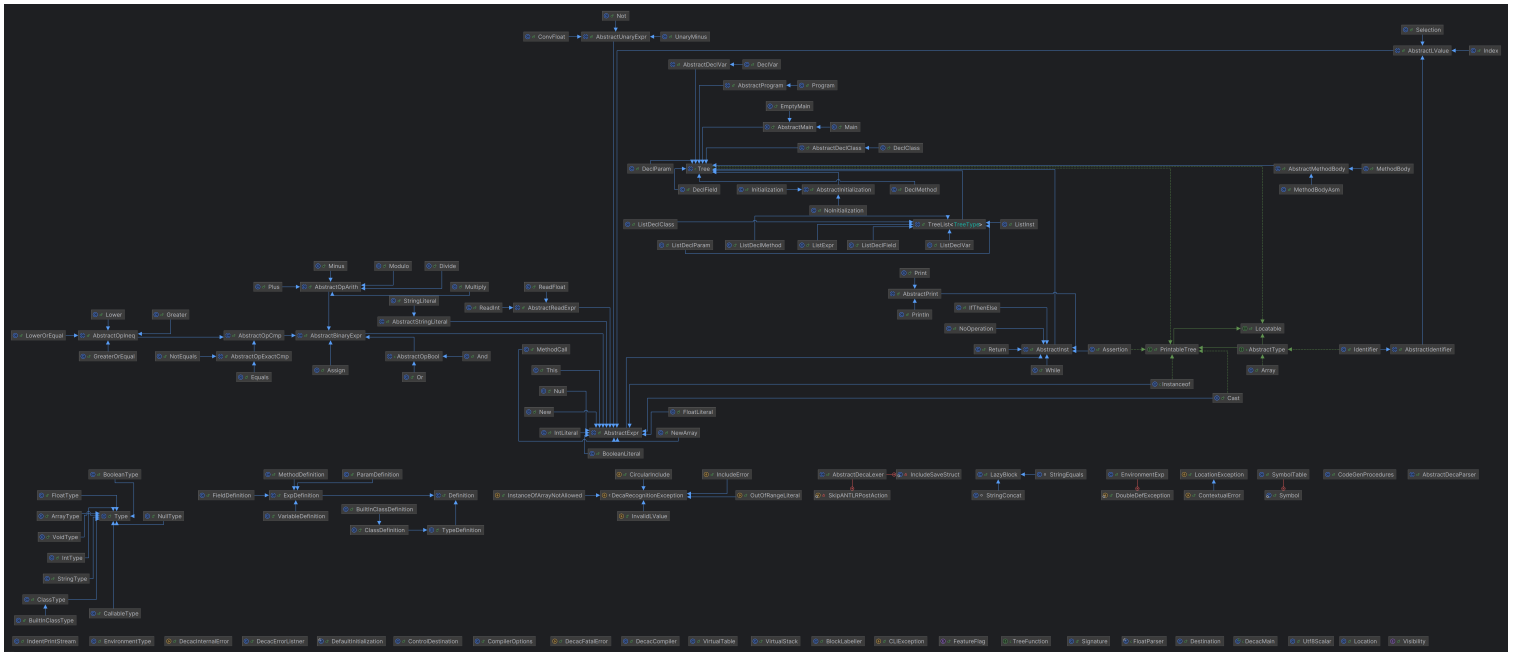
Conception du compilateur

À haut niveau, le compilateur prend entrée du code Deca, le valide, et produit le cas échéant un fichier assembleur pour l'interpréteur IMA.

JavaDoc disponible

Diagramme de classe

Ouvrir dans une nouvelle fenêtre



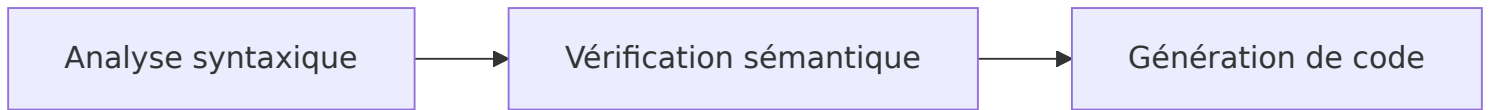
Composants

Pour ceci, le compilateur suit trois grandes étapes :

- La première est l'analyse syntaxique. Surnommée "étape A", elle exécute le lexer et le parseur ANTLR sur le code de l'utilisateur. Un arbre de syntaxe abstraite (AST) est ainsi construit. Il représente l'imbrication des blocs Deca.
- Si le code est syntaxiquement correct, sa sémantique est vérifiée. On entend par là prévenir de certains types d'erreurs à l'exécution en s'assurant de la cohérence générale

du programme.

- Une fois le programme vérifié, il est transcrit en langage assembleur attendu par IMA.



LazyBlock

Les LazyBlock sont une classe Java qui nous permet de coder un bout de programme IMA sans le placer dans la liste chaînée du compilateur, mais de connaître le label qui arrive sur ce block.

Ultérieurement, les LazyBlock sont appelés via leur méthode `codeGen` qui vient ajouter leur code à la liste d'instruction du compilateur.

Cette classe est utilisée pour les erreurs à l'exécution et certaines méthodes globales.

VirtualStack

L'un des besoins les plus récurrents des générateurs de code concerne l'affectation des registres. Chaque nœud de l'AST génère des instructions opérant sur des registres et doit être en mesure de savoir dans quels registres ses fils génèrent du code. Pour permettre au compilateur de tracer le moment où un générateur de code réserve un registre et le moment où il le libère, nous avons centralisé la gestion des registres dans la classe `VirtualStack`.

Ce système de pile virtuelle fournit un registre à tout générateur de code qui le demande. Cette classe attribue des registres tant qu'il en reste à disposition, et s'occupe de libérer temporairement un registre s'il n'y en a plus à disposition.

Gestion des registres

Le nom de la classe "pile virtuelle" vient du fait que les premières valeurs sont "empilées" sur des registres puis sur la pile en cas de débordement. Ainsi les 14 registres non-scratch sont privilégiés avant d'écrire dans une zone mémoire plus lente.

Nous avons opté pour une approche proche du Destination-Driven Code Generation¹ où le générateur de code prend en paramètre le registre où placer le résultat de l'expression. Nous limitons les déplacements mémoires grâce à ce fonctionnement.

Représentation

Nous représentons la propriété exclusive d'un registre avec la classe `Destination`. Elle correspond à un `new-type`. Une nouvelle classe est utilisée, car elle indique clairement qu'une valeur doit être placée par le générateur de code dans ce registre.

De plus, cette classe a un constructeur privé : seule la classe responsable des destinations peut créer des `Destination`. Cela nous assure que tant qu'aucun code n'accède à un registre non-scratch de lui-même, `VirtualStack` connaît tous les registres actuellement utilisés.

`VirtualStack` fournit la primitive d'allocation des registres via une lambda Java. Cela nous permet d'exploiter la pile d'appels Java comme un moyen de savoir sur quel intervalle le registre est alloué. En effet, cette forme d'API en callback permet d'insérer du code avant et après.

```
compiler.stack.scoped((Destination dest) -> {  
    // À partir d'ici, "dest" est réservé à cette expression  
    // Dans le cas où la classe VirtualStack a dû libérer un registre,  
    // elle a déjà sauvegardé la valeur précédente avec une instruction PUSH.  
    getRightOperand().codegenExpr(compiler, dest);  
    // À la fin de cette lambda, "dest" est libéré.  
    // Si on a dû précédemment libérer un registre, alors sa valeur précédente  
    // est restaurée.  
});
```

Gestion des débordements de pile

`VirtualStack` connaît les `PUSH` et les `POP` qu'elle utilise pour libérer des registres. Nous lui transmettons également les `ADDSP` et `BSR` faits dans les autres parties du code. Cette classe peut alors suivre le maximum utilisé de la pile et ainsi calculer `TST0` automatiquement.

Sauvegarde des registres

La class `VirtualStack` est en capacité de nous indiquer les registres utilisés par une méthode après son exécution.

On va donc utiliser cet indicateur pour sauvegarder les registres et les restaurer. À la génération du code, on va conserver la taille de la liste chaînée des instructions IMA du programme, cette valeur correspond à l'index où il faudra placer la sauvegarde des registres. À la fin de la génération de code de la méthode, on sait combien de registres sont utilisés. On peut donc ajouter les instructions `PUSH` à l'emplacement de notre sauvegarde et les instructions `POP` à la fin de la méthode.

On change également le comportement de l'instruction Déca `return` pour qu'à la place d'appeler l'instruction IMA `RTS` on réalise un branchement vers la restauration des registres qui elle implémente l'instruction `RTS`.

Label de saut

L'algorithme de destination driven code generation propose un codage des conditions comme des opérations sur le flot de contrôle. Ainsi la classe `ControlDestination` donne l'information aux générateurs de code qu'ils sont appelés pour un saut et peuvent ainsi améliorer la forme du code généré. Si le générateur de code n'est pas spécialisé, son implémentation par défaut est d'appeler le générateur de code dans le cas général et de faire un branchement en fonction de la valeur du booléen dans `Destination`.

Générateur de labels

La classe `BlockLabeller` est responsable d'uniformiser les labels générés. Le compilateur a besoin de différents labels pour générer des sauts. Pour cela, il a besoin de labels uniques. Ils seraient refusés par IMA sinon. Pour les rendre uniques, nous les suffixons d'un nombre incrémenté à chaque appel. Ce système nous permet de conserver le nom "if", "while" propre au label, et ainsi de lire plus facilement le code généré.

Nous avons noté que les identifiants Deca sont sensibles à la casse, tandis que les labels IMA ne le sont pas. Ainsi pour les labels générés à partir d'identifiants Deca comme les noms de méthodes, nous retirons les caractères invalides pour IMA. Pour éviter d'avoir un suffixe incrémental, notre approche a été ici de retenir tous les labels générés dans une `TreeMap`

Java. Elle nous permet de savoir si un label a déjà été utilisé (vérification insensible à la casse) et de ne suffixer les doublons que lorsque nécessaire.

Gestion des classes et des objets

Déclaration des classes

La déclaration des classes peut être source de nombreux problèmes. Nous utilisons ici une méthode en trois étapes pour palier les problèmes d'une gestion naïve.

3 étapes:

- La première consiste en la collecte des noms de l'ensemble des classes.
- La seconde est la déclaration des attributs. Nous initialisons ici chaque attribut à sa valeur par défaut : *0* pour *int* ou *null* pour les objets
- La troisième est la véritable initialisation comme attendue et spécifiée par l'utilisateur dans le programme. Cette initialisation se fait alors dans l'ordre de l'écriture.

Cela nous permet par exemple l'initialisation de *x* à *y* avant même la déclaration de *y*. Ici, *x* prendra alors la valeur par défaut de *y*.

Pour effectuer une gestion dynamique des appels de méthode, nous stockons dans l'espace mémoire de l'objet sa classe effective. (*ie Le compilateur connaît un type, mais l'objet peut être instance d'une classe plus précise.*) Cela se matérialise sous la forme d'une adresse vers la VTable de cette classe.

Table des méthodes (VTable)

Son but est de contenir les labels des méthodes. Nous pouvons y accéder grâce à un index propre à la signature de la méthode. Son principal atout est la simplification de l'héritage. Nous y stockons aussi en en-tête l'adresse vers la table du parent et vers une méthode *equals*

Lors d'un héritage, la Vtable de la nouvelle classe est une copie de celle du parent. Lors de la définition de méthode dans la nouvelle classe, il y a deux cas :

- Soit il s'agit d'une redéfinition : Une méthode surchargée possède la même signature et donc le même index dans la vtable que la méthode du parent. Le label est simplement écrasé par un nouveau vers la surcharge.
- Soit il s'agit d'une nouvelle méthode : Elle est ajoutée à la fin de la table.

Cast et Instanceof

Le calcul du booléen renvoyé par `instanceof` est réalisé à l'exécution. Le type réel d'un objet est connu dans l'en-tête de sa zone mémoire plus que par le compilateur. Le principe d'un `instanceof` est de savoir si l'objet (opérande de gauche) est lui-même égal ou possède un parent égale à la classe opérande de droite. Pour cela, une boucle est exécutée pour remonter les parents de l'opérande de gauche à la VTable. Le booléen est renvoyé lorsque l'égalité est trouvée ou lorsque l'adresse du dernier parent est *null* (adresse du parent de Object).

Le cast peut être abordé en plusieurs catégories :

- Le cast entre types primitifs : Conversion de *float* vers *int*, ou *int* vers *float*. Celui-ci est assez trivial, une instruction existe pour ce transtypage. Un cas particulier doit cependant être géré. Lors d'un cast de *float* vers *int*, il faut prendre en compte les valeurs minimal et maximal du type *int*.
- Le cast explicite pour un cast d'un type vers ce même type. (ex: `(boolean)(true);`)
- Le cast entre classe hérité est le plus problématique. Ici encore se distinguent deux cas
 - le cast d'une classe vers un de ces parents
 - Inversement, le cast d'une classe vers une classe plus basse dans l'héritage est complexe. Ce *downcast* demande une vérification lors de l'exécution. Pour cela, nous admettons à la compilation que le cast est possible. Cependant, une vérification est générée dans le code assembleur pour s'assurer que ce cast était effectivement valide. La vérification est similaire à l'algorithme de l'`instanceof` qui correspond très bien à la vérification d'un cast. Cela déplace donc le message d'erreur de la compilation à l'exécution.

Object

La classe Object est parent implicite de toutes les classes, elle permet d'utiliser la méthode `equals`.

L'implémentation de cette méthode est définie dans `CodeGenProcedures` avec un `LazyBlock` et compare uniquement si les adresses mémoires sont identiques.

Gestion des erreurs à l'exécution

Certains programmes Deca sont susceptibles de lever des erreurs à l'exécution. Un programme Deca n'a toutefois rarement besoin de générer le code de tous les gestionnaires d'erreurs possibles.

C'est dans l'objectif de facilement avoir accès aux labels de gestion d'erreurs et de ne les générer que si nécessaire que la classe `CodeGenProcedures` a été créée. Elle contient une liste de `LazyBlock`, qui sont des blocs de code qui ne sont générés que si le programme y accède lors de la génération de code.

Testeur automatisé

Le compilateur est accompagné d'un testeur automatisé écrit en Java qui permet de vérifier la validité des programmes Deca.

La classe `DecaTester` est le point d'entrée de ce testeur, qui vient parcourir le répertoire de tests et les classer automatiquement. Un `DecaTest` est la description d'un test découvert par le scanner de tests `DecaTestScanner`.

Nous avons également créé un `PrologueParser` qui permet de lire les commentaires des fichiers de tests afin d'extraire des informations supplémentaires sur les conditions du test.

Footnotes

1. [Destination-Driven Code Generation, 1990](#) ↩

Math

Math

Cette librairie fournit une variété de méthodes mathématiques qui traitent des variables de type `float`.

Fonctionnement

Pour utiliser les méthodes, il est nécessaire d'instancier un objet `Math` et d'inclure `"Math.decah"`.

Méthodes trigonométriques

Dans ces méthodes, un développement en série de Taylor spécifique à chaque fonction a été utilisé, ce qui permet de renvoyer une valeur précise (grâce à l'ULP).

En effet, pour chacune de ces méthodes trigonométriques :

- `float sin(float f)`
- `float cos(float f)`
- `float asin(float f)`
- `float atan(float f)`

La précision est obtenue grâce à une approche fondée sur l'utilisation d'une série de Taylor et un critère d'arrêt basé sur l'ULP (Unité au Dernier Rang). La fonction sinus, par exemple, est approximée à l'aide de son développement en série entière autour de zéro : `sin(x) = x - x3/3! + x5/5! - x7/7! + ...`

Chaque terme successif améliore l'approximation, mais devient de plus en plus petit. Cependant, dans un environnement à virgule flottante (ici, en 32 bits), il existe une limite à

la précision : en dessous d'une certaine valeur, les termes ajoutés n'ont plus d'effet réel sur le résultat final.

C'est là qu'intervient la méthode `ulp(f)` : dans chaque méthode, une boucle ajoute des termes à la somme tant que leur valeur absolue reste supérieure à l'ULP. Une fois qu'un terme devient plus petit que cette unité, on considère qu'il est numériquement négligeable et on interrompt le calcul.

Ainsi, l'approche adoptée ici ne dépend pas d'un nombre fixe d'itérations, mais d'un seuil d'arrêt dynamique basé sur la précision réelle du `float`.

`float ulp(float f)`

La méthode `ulp` calcule la plus petite variation possible pour une valeur flottante donnée. Elle mesure le pas de discrétisation du nombre flottant à l'échelle de `f`, grâce à l'exploration de son exposant sous forme 32 bits.

En plus des méthodes trigonométriques et de `ulp`, d'autres méthodes ont été ajoutées, en s'inspirant de la librairie standard de Java, parmi lesquelles :

-
- `float toDegrees(float rad)` : Convertit un angle en radians en degrés avec la formule $\text{rad} \times 180 / \pi$.
 - `float ln(float a)` : Approxime le logarithme népérien à l'aide de la série de Taylor de $\ln((1 + x)/(1 - x))$, où $x = (a - 1)/(a + 1)$, convergence rapide pour `a > 0`, arrêtée selon l'ULP.
 - `float exp(float x)` : Calcule e^x via une série de Taylor : $1 + x + x^2/2! + x^3/3! + \dots$, jusqu'à ce que le terme courant soit négligeable selon l'ULP.
 - `float pow(float a, float b)` : Implémente a^b pour des exposants entiers (positifs ou négatifs) via des multiplications successives (cas général non géré).
 - `float abs(float a)` : Renvoie la valeur absolue d'un `float` : retourne `a` si positif, `-a` sinon.
 - `float sqrt(float f)` : Approxime la racine carrée par l'algorithme de Newton-Raphson : itère $x = (x + f/x)/2$ jusqu'à convergence.

- `float exp32bits(float f)` : Estime l'exposant en base 2 d'un `float f` en le multipliant ou divisant par 2 jusqu'à ce que `f` soit dans `[1, 2)`.
 - `float floor(float a)` : Renvoie la partie entière inférieure d'un `float` en décrémentant par pas de 1 jusqu'à atteindre la plus grande valeur $\leq a$.
 - `float ceil(float a)` : Renvoie le plus petit entier $\geq a$, en appelant `floor(a)` puis en l'ajustant si nécessaire.
 - `float normalize(float f)` : Ramène un angle `f` dans l'intervalle `[- $\pi$ ,  $\pi$ ]` en retranchant des multiples de `2 $\pi$` à l'aide de `floor()`.
 - `int fact(int n)` : Calcule récursivement la factorielle d'un entier `n` : `n! = n × (n-1)!`, avec arrêt à `n = 1`.
-

Attributs de Math

- `float pi = 3.1415927;`
 - `float two_pi = 2 * this.pi;`
 - `float MIN_FLOAT = this.pow(2, -149);` Correspond à la plus petite valeur positive normale que peut représenter un float 32 bits (le plus petit pas possible entre deux nombres flottants).
-

Classes Beziers

Bezier

La classe `Bezier` représente une **courbe de Bézier générique** et sert comme une **classe de base (abstraite)** pour modéliser des courbes définies par un ensemble de points de contrôle.

LinearBezier

La classe `LinearBezier` est une **sous-classe de `Bezier`** qui représente **un segment de droite** entre deux points.

Point

La classe `Point` représente un **point dans l'espace** qui est présenté grâce:

- Des coordonnées (float)
- Un angle

Cette classe est utilisée pour définir les points de contrôle des courbes de Bézier.

Paint

La bibliothèque Paint fourni un programme pour dessiner sur le terminal.

Elle repose sur l'extension `UserIO` et `Math`.

Choix de conception

À l'heure actuelle, l'extension paint est limitée à une sélection de 4 points maximum, car elle a été créée avant d'avoir l'extension ajoutant les tableaux.

Paint a été créé par ajout successif de méthodes sans réusinage du code pour s'assurer d'avoir une version toujours fonctionnelle livrable.

Il n'y a pas de tests automatisés créés par manque de temps et de complexité de tester une boucle infinie et le contenu affiché dans un terminal ansi que le comportement différents selon différents terminaux.

Problèmes connus

Certains dessins ne sont pas parfait à cause d'approximation de flottant en coordonnées décimale.

Certain clic sur la souris ou clavier s'écrivent dans le terminal. Un fix pour en éviter la majorité a été ajouter, mais n'est pas parfait, voici son fonctionnement lors d'un évènement :

```
Déplacer le cursor à l'endroit souhaité  
dessiner  
replacer le curseur en 0,0  
réécrire la premiere ligne\
```

Dans la majorité des cas, le texte non souhaité s'écrit pendant que le curseur est en 0,0. Vu qu'on réécrit la ligne ce n'est pas gênant.

Mais parfois le texte s'écrit au milieu du tableau.

Détails d'implémentation

Le tracé des lignes repose sur la bibliothèque math qui propose les formules pour les courbes et droites de bezier.

Le remplissage d'une forme repose sur l'algorithme du `Ray-Casting`

UserIO

La bibliothèque UserIO fournit des méthodes pour échanger des informations avec le terminal.

Pour cela, elle utilise les fonctionnalités proposées avec le standard `ANSI` et les instructions IMA `WUTF8` et `RUTF8`.

Le standard `ANSI` permet d'échanger des messages sous la forme :

| `ESC[xxxxx`

Où xxxx est le code associé. En fonction des codes, on peut activer les inputs de la souris, changer la couleur du fond, modifier la position du curseur, etc.

La bibliothèque intègre un certain nombre de méthodes implémentées en assembleur pour écrire directement dans le terminal.

Mécanique

La bibliothèque UserIO repose sur un système de callback. La classe UserIO a une méthode principale `start()` qui écoute en permanence les entrées utilisateur et les informations du terminal, et en fonction des entrées, appelle les différentes méthodes que le développeur aura remplies en créant sa classe qui étend UserIO.

Modification du langage Déca et du compilateur

Aucun ajout au Lexer et Parser ni à l'AST n'a été fait.

Les instructions `WUTF8` et `RUTF8` ont été implémentées via des méthodes en assembleur directement pour ne pas modifier le code Java. Ça permet d'être plus efficace et de ne pas avoir à gérer un flag d'activation de ces instructions.

Tableaux

Du point de vue de la mémoire, un tableau est un espace mémoire réservé, contenant des données contiguës. L'emplacement réservé prend donc `octets par mot * taille du tableau`. Une variable contenant un tableau contient en réalité un pointeur vers la première case du tableau.

Changements dans la grammaire

Pour ajouter les tableaux au parser, il a été nécessaire de modifier la grammaire.

Dans les expressions primaires, nous avons ajouté des règles à celles déjà existantes:

```
primary_expr ::= ... \  
              | NEW type '[' expr ']' \  
              | primary_expr '[' expr ']
```

Ces règles nous permettent l'initialisation de tableaux et l'accès par index aux éléments des tableaux.

Afin que le parser considère les tableaux comme un type, nous avons également ajusté la règle `type`:

```
type ::= ident ('[' ''])*
```

Ainsi, chaque identifiant rencontré a la possibilité d'être vu comme un tableau, voire même un tableau à plusieurs dimensions en ajoutant plusieurs '[']'.

Gestion de l'initialisation

Les éléments des tableaux initialisés ont une valeur par défaut:

- Tableaux de `int` et de `float` : `0` et `0.0` respectivement

- Tableau de `boolean` : `false`
- Tableaux d'objets, de tableaux ou de `string` : `null`

Cast et instanceof

L'utilisation de l'instruction `instanceof` et des casts sur un tableau sont interdits.

Ce choix est dû au fait que dans le cas d'un tableau d'objet, ce tableau peut potentiellement contenir plusieurs objets de différentes classes à cause de l'héritage.

Il n'existe par ailleurs pas de type `array` visible par l'utilisateur.

Print avec les tableaux

Il n'est pas possible d'utiliser `print` et ses dérivés avec les tableaux. C'est un choix, étant donné que cela est impossible en Java, et que Deca est un mini-Java, nous avons décidé de ne pas supporter le `print` avec un tableau.

IMA

Nous avons rajouté l'instruction IMA `RegisterIndex` à notre génération de code. Elle nous permet d'accéder à l'index d'un tableau en utilisant un registre comme pointeur vers la première case du tableau, et un autre registre contenant le nombre de mots de décalage pour atteindre la case souhaitée

Messages d'erreurs

Affichage des messages d'erreurs contextuelles

L'affichage des messages d'erreurs avec cette extension affiche la ligne du code `deca` qui cause l'erreur contextuelle, tout en colorant en rouge la partie exacte qui représente la source d'erreur.

Choix de conception

La classe `LocationException` dans le package `tree` dispose d'une méthode `display` qui offre une façon générique d'afficher les messages d'erreurs.

Pour afficher la ligne du code `deca` contenant l'erreur, on utilise l'attribut `location` pour lire le fichier à compiler et récupérer la ligne en question.

Afin de pouvoir colorer seulement la source d'erreur, il fallait aussi déterminer la fin de l'expression causant l'erreur. Pour cela, on a ajouté un paramètre `endToken` dans la méthode `tokenLocation` utilisé par le parseur pour définir la *location* d'une expression.

Dans la classe `Location`, on a ajouté deux attributs, un pour stocker l'index de fin de l'expression, afin de délimiter la partie du code `deca` qui cause l'erreur et un deuxième pour stocker l'indice de la ligne de fin pour pouvoir gérer le cas des erreurs sur des multiples lignes.

String

Contrairement à des formats de binaire comme l'ELF, la machine virtuelle IMA ne dispose pas d'une zone mémoire pour stocker des valeurs statiques.

Représentation mémoire

Nous considérons les chaînes de caractères comme un tableau de scalaires UTF-8, avec comme toute première valeur la taille du tableau. La chaîne "hello" est donc encodée comme une allocation de 6 mots : `5 | 104 | 101 | 108 | 108 | 111`.

Nous avons choisi de sauvegarder la taille directement plutôt que de terminer la chaîne de caractères par un octet nul puisque les algorithmes écrits dépendent d'un accès rapide à cette taille.

Les chaînes de caractères sont systématiquement allouées sur le tas. Les littéraux sont générés comme des `LOAD` de chaque valeur UTF-8 successive dans un nouvel objet.

Opérations sur les chaînes de caractères

Affichage

L'affichage d'un littéral chaîne de caractères est optimisé pour faire appel à l'instruction `WSTR`. Dans le cas des chaînes dynamiques, une boucle est générée pour itérer sur tous les scalaires UTF-8 et utiliser l'instruction `WUTF8`.

Concaténation

La concaténation de chaînes de caractères s'effectue pour l'utilisateur comme une addition, mais elle génère un appel de procédure en arrière-plan. Le code assembleur généré étant

conséquent, déplacer cette procédure dans une fonction permet de réduire la taille du code final.

Nous additionnons la taille des deux chaînes de caractères pour allouer la chaîne de caractères résultante. L'algorithme copie ensuite les scalaires UTF-8 de la première chaîne puis de la deuxième.

Égalité

La méthode `equals` du type `string` est une pseudo-méthode (car ce n'est pas la méthode `Object` et elle n'utilise pas de *virtual table*). Elle compare les longueurs des deux chaînes, et si elles sont identiques, vérifie l'égalité scalaire UTF-8 par scalaire.

Longueur

La méthode `length` est aussi une pseudo-méthode. Elle récupère simplement le premier mot de la zone mémoire de la chaîne de caractères.

Modification du système de types

Nous avons cherché à minimiser les changements sur notre compilateur pour conserver un compilateur respectant optionnellement la spécification Deca originale.

Ainsi, le drapeau `-fstring-object` vient ajouter le type `string` dans l'environnement des types accessibles à l'utilisateur. Ce même drapeau active dans `StringType` une modification de son comportement lorsque le type est comparé avec `null`.